GeCCo Presentation Script — Slides 1–7

Full-Sentence Version · ~10 minutes total · Rmus et al. (2025)

Slide 1 – Opening

≈1 min

Good [morning/afternoon], everyone. Today we're presenting our reading of Rmus et al., 2025 — a NeurIPS paper introducing GeCCo, a pipeline for generating computational cognitive models using large language models. I'm Yingzhu Chen, and this is Christian Block. Before we start, a quick note on how we're approaching this talk: we're not going to give you a complete summary of every section. Instead, we want to walk you through an argument — what the paper claims, what evidence supports it, and where we think the claim needs to be qualified. Our core message for today is this: LLMs can meaningfully accelerate the discovery of cognitive models, but a better statistical fit is not automatically the same thing as a better psychological explanation. To understand why that distinction matters, we first need to look at why cognitive modeling is difficult in the first place.

Slide 2 – Problem

≈1.5 min

Traditionally, building a cognitive model looks like this: a researcher starts with a theory, translates it into code, fits that code to behavioral data, and then revises the theory based on how well it fit — and repeats this cycle, often many times. This is slow, and it requires a rare combination of skills: you need to be a domain expert in cognitive science, a theoretician, and a software engineer, all at once. But the deeper issue the paper points to isn't just speed — it's that this process tends to produce a narrow model space. Researchers naturally gravitate toward theories they already know how to formalize and code, which means promising alternative explanations may simply never get tested. So the motivation for GeCCo isn't just 'let's do this faster' — it's 'let's widen the space of models we even consider.' With that motivation in mind, let's be precise about what we mean by a 'cognitive model' in this context.

Slide 3 – Concept

≈1.5 min

A computational cognitive model is essentially a psychological idea that has been written down as equations or executable code, so that it can generate testable predictions. Take a simple idea like 'people learn from rewards.' In this paper's framework, that idea becomes a Python function — something like this template you see here, with a learning rate and an inverse-temperature parameter — and that function outputs predicted choice probabilities that can be directly compared to real human behavior. One important reason we bother comparing multiple models at all is that very different psychological mechanisms can sometimes produce very similar surface-level behavior. So you can't just look at behavior and know the mechanism — you need to fit competing

models and see which one explains the data better. GeCCo's whole contribution is to automate part of this model-building process. Let's look at how.

Slide 4 – Method: the loop

≈1.5 min

At its core, GeCCo is a loop, and it's worth holding this loop in your head for the rest of the talk. First, the LLM receives a prompt containing the task description and some behavioral data. Based on that, it writes three candidate model codes. Those three candidates are then fit to a separate, held-out portion of the data — data the LLM never saw. The best of the three is selected using BIC, the Bayesian Information Criterion. And finally, information about that best model is fed back into the prompt for the next round, so the LLM can build on what worked and avoid repeating itself. The important thing to notice here is that the LLM is not doing the evaluation — it's proposing candidates. The actual selection is done by fitting to data and computing BIC. The authors describe this as 'guided code generation,' explicitly distinguishing it from just asking an LLM once and taking whatever it gives you. Given how central this loop is, it's worth looking more closely at what actually goes into that prompt.

Slide 5 – The prompt as method

≈1.5 min

The prompt in GeCCo isn't a casual, one-line request — it has five distinct ingredients: a task description explaining what participants actually did, trial-by-trial behavioral data, guardrails specifying exact function names and input-output formats, a code template that acts as a syntactic scaffold, and, from the second iteration onward, feedback about the best model found so far. The reason this matters is that prompting here is functioning as part of the experimental method — it's constraining what the LLM can output, so that every generated model is executable and directly comparable to the others. But that control comes at a cost: more constraints can mean less room for genuinely novel, free-form model structures, and it opens the door to what the authors themselves call template bias — the scaffold nudging the LLM toward certain model shapes. Once these models are generated and fitted, though, how does the paper actually decide which one is 'good'?

Slide 6 – Evaluation

≈1.5 min

The paper uses three complementary tools, and I want to give you the intuition for each rather than the formulas — those are available in our backup slides if anyone wants them later. NLL, or negative log-likelihood, asks: how surprised is the model by the choices people actually made? Lower is better. BIC takes that same fit but adds a penalty for extra parameters, so it rewards models that explain the data well without being needlessly complex. And PPC, posterior predictive checks, asks a slightly different question: if you simulate behavior from the fitted model, does it actually look like real human behavior? Here's the key point we want you to take away from this slide: a lower BIC tells you one model compares favorably to another — it does not, by itself, prove that the model

captures the true underlying psychological mechanism. That distinction is going to become important later in our critical evaluation. For now, let's see how this evaluation approach performed once it was actually tested.

Slide 7 – Evidence across four domains

≈1.5 min

The authors tested GeCCo across four quite different cognitive domains, each with its own established literature baseline: decision making, compared against a baseline called pWADD; learning, compared against a four-learning-rate Rescorla-Wagner model; planning, compared against a Hybrid model-based/model-free system; and working memory, compared against the RL-WM model. Across all four, the headline result is that LLM-generated models matched or outperformed these literature baselines — that's a fairly strong claim of generality. We do want to flag one nuance here, though: in the planning domain, the LLM-generated model was favored by the exceedance probability, but the BIC difference itself wasn't statistically significant — so the result there is more 'competitive' than 'clearly superior.' We're not going to walk through all four experiments in equal depth. Instead, we'll use decision making and learning as our two representative deep-dive cases, and then summarize planning and working memory more briefly, so we can get to the parts of the paper that need real critical scrutiny.

Timing note: this script runs to roughly 1,350–1,450 spoken words, landing at about 10 minutes at a natural pace (~135–145 wpm).